



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure
Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width - Width of the grid world
 self.height- Height of the grid world
 self.walls - Location of obstacles that block movement
 self.food_positions - Position of all food items available for collection
 self.opponents - position of the adversarial agent
 self.agent_pos - the current position of the agent within the environment.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi Agent Environment
 The environment contains the main agent and one or more autonomous opponent agents that have their own states and actions, making it a multi-agent system, even if num_opponents is set to 0 in a particular run.so thats why the environment is multi agent

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all percepts an agent has received from the environment. It provides the information the agent uses to choose its actions

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

`get_percept()` method represents the Sensors component

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Its Partially Observable

Because the agent does not have access to the complete state of the environment. It receives only limited information through `get_precept()` It cant directly get the position of all food items or walls

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Actively updated component is the Performance Measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Its important because the performance measure should assess the results of the agents actions in external environment. Penalizing wall collisions evaluates how effectively the agent navigates the grid and achieves its goals, whereas measuring internal processing effort would evaluate the implementation of the agent rather than its success in interacting with the environment

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Adding `self.toxic_traps` while hiding it from `get_percept()` keeps the environment Partially Observable because the agent cannot perceive all relevant environmental information that influences its performance and decisions

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos)` in `self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'` , you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Therefore `smells_toxin` improves rationality by giving the agent more accurate knowledge of the environment, allowing it to select actions that are more likely to achieve a higher utility. It does not make the agent automatically rational; it only provides the information needed for rational decision making. The agent's action strategy must still use this information effectively

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World Exploit demonstrated that a poorly designed metric can make an agent optimize the score instead of the true objective. A rational agent will maximize the given performance measure, so if the metric is wrong, the agent may find a loophole and behave undesirably

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING